

INF264 PROJECT 2: Gift Recognizer 4 Santa

Members:

Lyder Samnøy

Marius Reikerås

Division of labour

Marius coded for task 1 and 3, and wrote the rapport for task 3. Lyder coded task 2 and wrote most of the rapport. We also consulted each other for our own tasks.

Introduction

This Christmas themed task aims to explore different machine learning classifiers in situations where data is both labeled and unlabeled. As well as how PCA is used to drastically lower the number of features in a data set without compromising on the accuracy of the models.

Data Description

We make use of two datasets; `dataset.npz` for task 1 and 2, and `dataset_corrupted.npz` for task 3. `dataset.npz` is a one-dimensional array of grayscale values, with 2614 images in total, corresponding to 14 different classes of labeled objects. `dataset_corrupted.npz` is smaller, with only 405 images, these are not labeled, and 89 of the images are "corrupted", meaning they should not conform to any of the image classes.

Task 1: Automatic Gift Recognizer

In this task, we aim to find reliable classifiers to correctly identify the images. We chose three different models, K nearest neighbors(KNN), Random Forest(RF) and Support Vector Machine(SVM), comparing them and seeing which is more accurate, also comparing other factors like runtime.

K nearest neighbors was chosen because it is a simple and easy to understand model, where noise levels can be easily tweaked by incrementing the `n-neighbors` parameter. With Random Forest, we wanted a more sophisticated model that utilizes several models (individual decision trees and the ensemble forest) to see if light ensemble learning yields a higher accuracy. SVM was added to test how a model that finds clear boundaries between classes performs, and because it can handle more complex shapes in the dataset.

We used Scikit-learn for all models, including its tools for testing different parameters and measuring results. After comparing the models, SVM gave the best accuracy, while KNN was the fastest to train. Random Forest performed well overall but was slightly less accurate than SVM. Because of this, we used SVM again in Task 3 to help detect corrupted images.

Here are the best results for each classifier: Random Forest = 0.725 KNN = 0.757 SVM = 0.792

We see that the SVM classifier yielded the highest accuracy score. Our SVM uses the RBF kernel (Radial Basis Function). The RBF kernel maps high-dimensionality data. This fits our use-case perfectly, as image data is often characterized by complex non-linear relationships between pixels. It's worth noting that the SVM had a significantly higher runtime than the other classifiers, and we had to limit the training set to get it to a manageable level.

KNN performed the second best, likely because KNN is good at highlighting similarity in data. As the images in the same image-class share a lot of the same feature space, KNN effectively clusters these images together.

Random Forest likely performed worst because Random Forest models don't take pixel relationships into account (capturing curves and edges). Instead, each pixel is treated as an independent feature. This problem is compounded by the large amount of features in each image (400), as we intentionally didn't perform any dimensionality reduction on the dataset in this task. It's worth noting that some of the image classes (class 3 and 7) represented large dips in accuracy across all classifiers, but this was particularly pronounced in the Random Forest classifier, reducing overall accuracy score.

Task 2 Dimensionality Reduction

Task 2 aims to explore how Principal Component Analysis (PCA) can be used to reduce dimensionality in data without compromising on accuracy. For our dataset, the default feature number is 400. In this task, we aim to reduce the feature number, and analyze how the accuracy score is impacted.

We used Sklearn's existing PCA implementation with our best parameters from task 1. We found that the accuracy stayed roughly the same at higher k-values, with a double-digit % decrease in accuracy for both models only at k-values 8 and below. This illustrates how PCA can drastically reduce the number of features without affecting the accuracy. PCA sorts the features by variance. So in our case, when f.ex. $k = 10$, only the 10 pixels that generate the most variance are kept. Often, this is enough to make an equally accurate prediction compared to having the full feature set, as the majority of features are redundant. It's worth noting as well that changes in accuracy that are below 10% are probably margin of error.

Task 3: Bad Data

In this task, we were given the dataset `corrupted_data.npz`, containing 405 unlabeled images where 89 are known to be corrupted. These corrupted images do not belong to any of the 14 known categories. The goal of this task was to automatically identify the corrupted images. We approached this as an unsupervised anomaly detection problem, where the goal is to find images that differ from the original dataset categories from Task 1. As the task description suggested, we reused our model from Task 1 the SVM, since it achieved the best performance, and combined it with PCA and KMeans clustering. The idea was for the SVM model to measure model uncertainty for each image, where higher uncertainty indicates potential anomalies. We then combined this with PCA, which calculated a reconstruction error that measures how well each image fits the general structure of the "normal" data. A higher reconstruction score indicates higher noise or corruption. After normalizing both values, we used KMeans clustering to group the images into two clusters: one representing normal images and one representing corrupted ones. The cluster with the lowest average model confidence was labeled as "corrupted". This approach flagged 88 out of 405 images as corrupted. Since there were no labels to verify the predictions directly, we manually inspected the flagged images. Almost all of them appeared visibly corrupted or noisy, suggesting that the approach worked very well. We therefore estimate

the method's accuracy to be approximately 88/89, matching closely with the known number of corrupted images in the dataset.